00 01 02 03 04 05 06 **07** 08 09 0A 0B 0C 0D

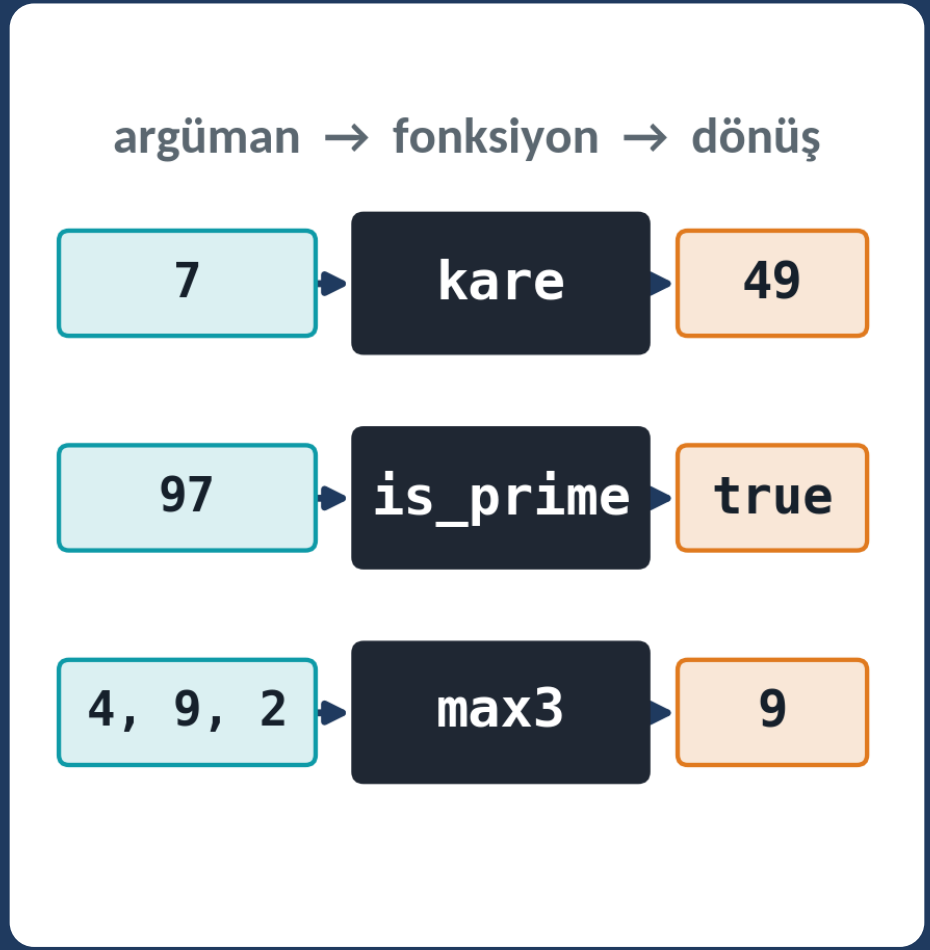
ALGORİTMA VE PROGRAMLAMA – I

HAFTA 08

Fonksiyonlar I: Modüler Programlama ve Soyutlama

Prototip, tanım ve çağrı · değerle aktarım · sözleşme ve kara kutu · standart kütüphane · yukarıdan aşağıya tasarım · assert ile birim testi

MODÜL IV · MODÜLER PROGRAMLAMA



Dr. Öğr. Üyesi Mehmet Sevri · RTEÜ Bilgisayar Mühendisliği · 2026–2027 Güz

14 HAFTALIK YOLCULUK

Yol haritasında neredeyiz?

H1 Bilgisayar ve Program	**H2** Algoritmik Düşünme	**H3** Değişkenler ve Veri Tipleri	**H4** Operatörler ve İfadeler	**H5** Karar Yapıları	**H6** Döngüler I	**H7** Döngüler II + Ara Proje
0x00	0x01	0x02	0x03	0x04	0x05	0x06
H8 **Fonksiyonlar I**	**H9** Kapsam, Yığın, Özyineleme	**H10** Diziler I	**H11** Diziler II: Matrisler	**H12** İşaretçiler ve Bellek	**H13** Karakter Dizileri	**H14** Yapılar, Dosyalar, Final
0x07	0x08	0x09	0x0A	0x0B	0x0C	0x0D

Vize: H1–H7 · Ara proje: H7 · Final projesi: H14

GERİ ÇAĞIRMA · H7

Geçen haftadan · S1: Program ne yazdırır?

quiz/H08_Q01_ic_ice_sayac.c · satır 10–16

```
10      int sayac = 0;
11      for (int i = 1; i <= 4; i++) {
12          for (int j = i; j <= 4; j++) {
13              sayac++;
14          }
15      }
16      printf("%d\n", sayac);
```

A 16

B 10

C 6

D 4

TAHMİN ET

Geçen haftadan · S2 ve S3

S2 (√n asallık): 97 için iyileştirilmiş test kaç bölen dener? Naif test (2..96) kaç?

quiz/H08_Q02_karekok_bolen.c · satır 11–19

```c
    int n = 97, denenen = 0, asal = 1;
    for (int d = 3; d <= n / d; d += 2) {
        denenen++;
        if (n % d == 0) {
            asal = 0;
            break;
        }
    }
    printf("denenen bölen: %d, asal mı: %d\n", denenen, asal);
```

S3 (menü): Girdi `1 2 1 3` ise çıktı? `switch`'teki `break` döngüyü bitirir mi?

quiz/H08_Q03_menu_break.c · satır 12–26

```c
    do {
        if (scanf("%d", &secim) != 1) {
            return 1;
        }
        switch (secim) {
        case 1:
            printf("A");
            break;
        case 2:
            printf("B");
            break;
        default:
            break;
        }
    } while (secim != 3);
```

Yanıtlar: gerçek çıktılar

S1 · B: 10

GERÇEK ÇIKTI

```
$ ./q01
10
```

- İç döngü `j = i`'den başlar: 4 + 3 + 2 + 1
- n(n + 1)/2 sezgisi

S2 · 4 bölen

GERÇEK ÇIKTI

```
$ ./q02
denenen bölen: 4, asal mı: 1
```

- d = 3, 5, 7, 9; 11 > 97 / 11 = 8 → dur
- Naif test: **95** bölen

S3 · ABA bitti

GERÇEK ÇIKTI

```
$ ./q03
1 2 1 3
ABA bitti
```

- `break` yalnızca `switch`'ten çıkar
- Döngüyü `while (secim != 3)` bitirir

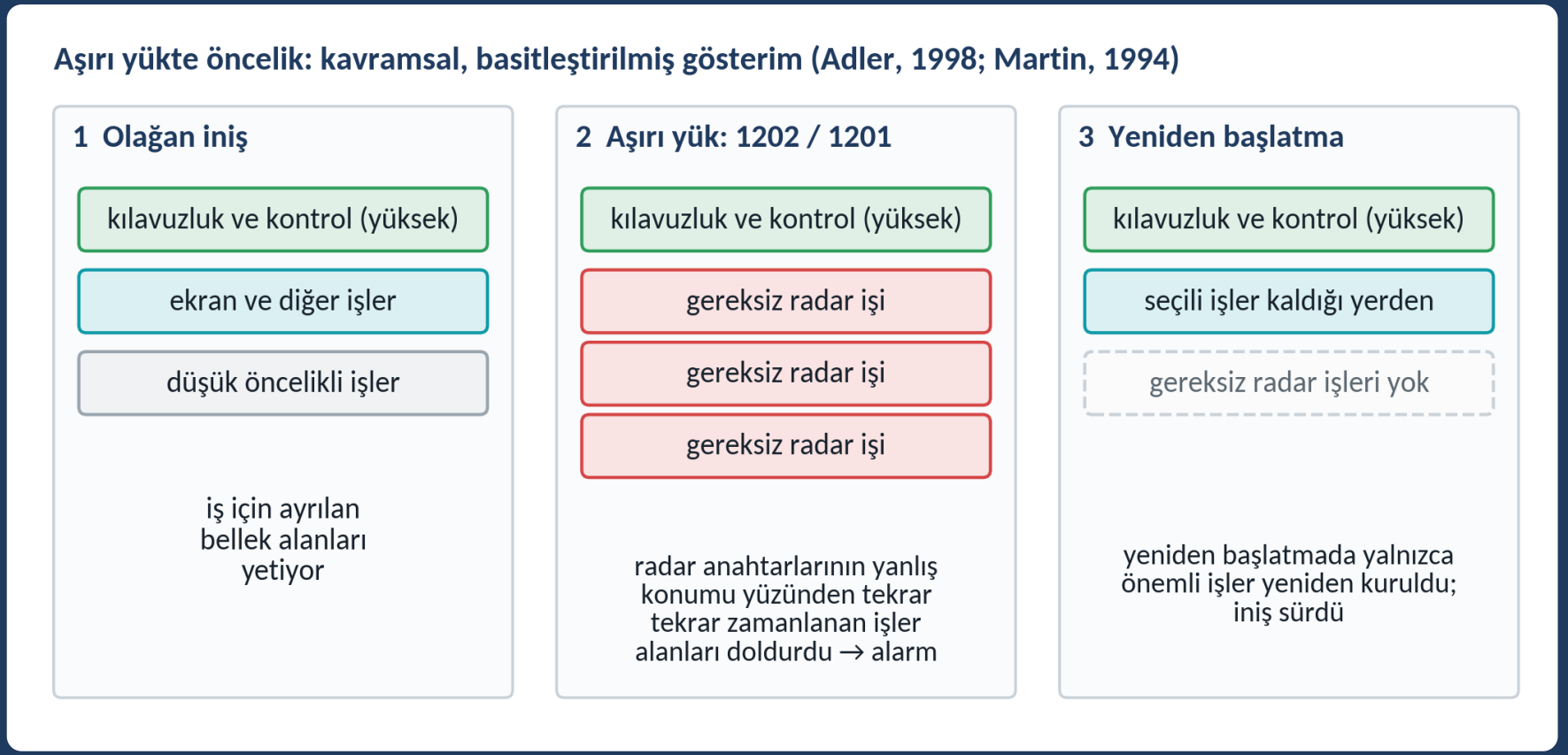
H7'de bu kodlar **main'in içindeydi**. Bugün asallık testi `is_prime(n)`, ATM programı ise ayrı ayrı test edilebilen **fonksiyonlar** olacak.

GERÇEK DÜNYADAN

20.07.1969

Apollo 11: “Program alarm… 1202”

- İnişin son dakikaları: **1202** ve **1201** alarmları
- Gereksiz **radar işleri** iş alanlarını doldurdu
- Yazılım yeniden başladı; **öncelikli işler** sürdü
- Görev Kontrol: devam → Kartal Ay’a kondu
- Uçuş yazılımı: **Margaret Hamilton**, MIT



Jones & Glover (t.y.) · Adler (1998) · Martin (1994) · Computer History Museum (t.y.) · NASA Science (2024)

BÜYÜK SORU

Haftanın Büyük Sorusu

Milyonlarca satırlık yazılımlar, hiçbir mühendis tamamını aklında tutamadığı hâlde nasıl geliştirilir ve doğrulanır?



Böl

Problemi **adı konmuş**, tek iş yapan parçalara ayır (yukarıdan aşağıya).



Sözleşme

Parçayı kullanan yalnızca **ne** yaptığını bilir; **nasıl** yaptığını değil.



Doğrula

Her parçayı **ayrı** test et; birleşince testleri yeniden çalıştır.

H7'nin ATM'si: **144 satır, tek main**, girdi bloğu 4 kez → bugün: **main + 8 fonksiyon**, çıktı `diff` ile birebir aynı

ÖĞRENME ÇIKTILARI

Bu hafta sonunda neler yapabileceksiniz?

ÖÇ 8.1	Fonksiyon tanımı, prototipi ve çağrısı sözdizimini doğru kullanır.
ÖÇ 8.2	Değerle aktarım (pass by value) mekanizmasını açıklar ve sonuçlarını öngörür.
ÖÇ 8.3	Bir problemi yukarıdan aşağıya adım adım iyileştirme (stepwise refinement; Wirth, 1971) ile alt görevlere ayırır ve her birini fonksiyon olarak gerçekler.
ÖÇ 8.4	Standart kütüphane fonksiyonlarını (math.h, ctype.h, stdlib.h, time.h) belgelerinden okuyarak kullanır.
ÖÇ 8.5	assert ile fonksiyonlar için basit birim testleri yazar.

01

Fonksiyonlar: tanım, çağrı ve değerle aktarım

neden fonksiyon? · kara kutu · anatomi · prototip · void ve () · çağrı akışı · değerle aktarım

Neden fonksiyon? Formülü bir kez yaz, adıyla çağır

Fonksiyonsuz: formül 3 kez

H08_K01_tekrar_fonksiyon.c · satır 14–17

```
14      /* 1) Fonksiyonsuz: aynı formül üç kez */
15      printf("Ali  : %.2f\n", 0.40 * 70 + 0.60 * 85);
16      printf("Ayşe : %.2f\n", 0.40 * 90 + 0.60 * 95);
17      printf("Can  : %.2f\n", 0.40 * 45 + 0.60 * 48);
```

Fonksiyonlu: formül tek yerde

H08_K01_tekrar_fonksiyon.c · satır 19–22

```
19      /* 2) Fonksiyonlu: formül TEK yerde, adıyla çağrılır */
20      printf("Ali  : %.2f\n", agirlikli_ortalama(70, 85));
21      printf("Ayşe : %.2f\n", agirlikli_ortalama(90, 95));
22      printf("Can  : %.2f\n", agirlikli_ortalama(45, 48));
```

H08_K01_tekrar_fonksiyon.c · satır 26–30

```
26  /* %40 vize + %60 final (Mini-OBS kuralı). */
27  double agirlikli_ortalama(double vize, double final)
28  {
29      return 0.40 * vize + 0.60 * final;
30  }
```

GERÇEK ÇIKTI: İKİ BLOK AYNI

```
$ ./tekrar_fonksiyon
Ali  : 79.00
Ayşe : 93.00
Can  : 46.80
Ali  : 79.00
Ayşe : 93.00
Can  : 46.80
```

Soyutlama: ne yaptığını bil, nasılını değil

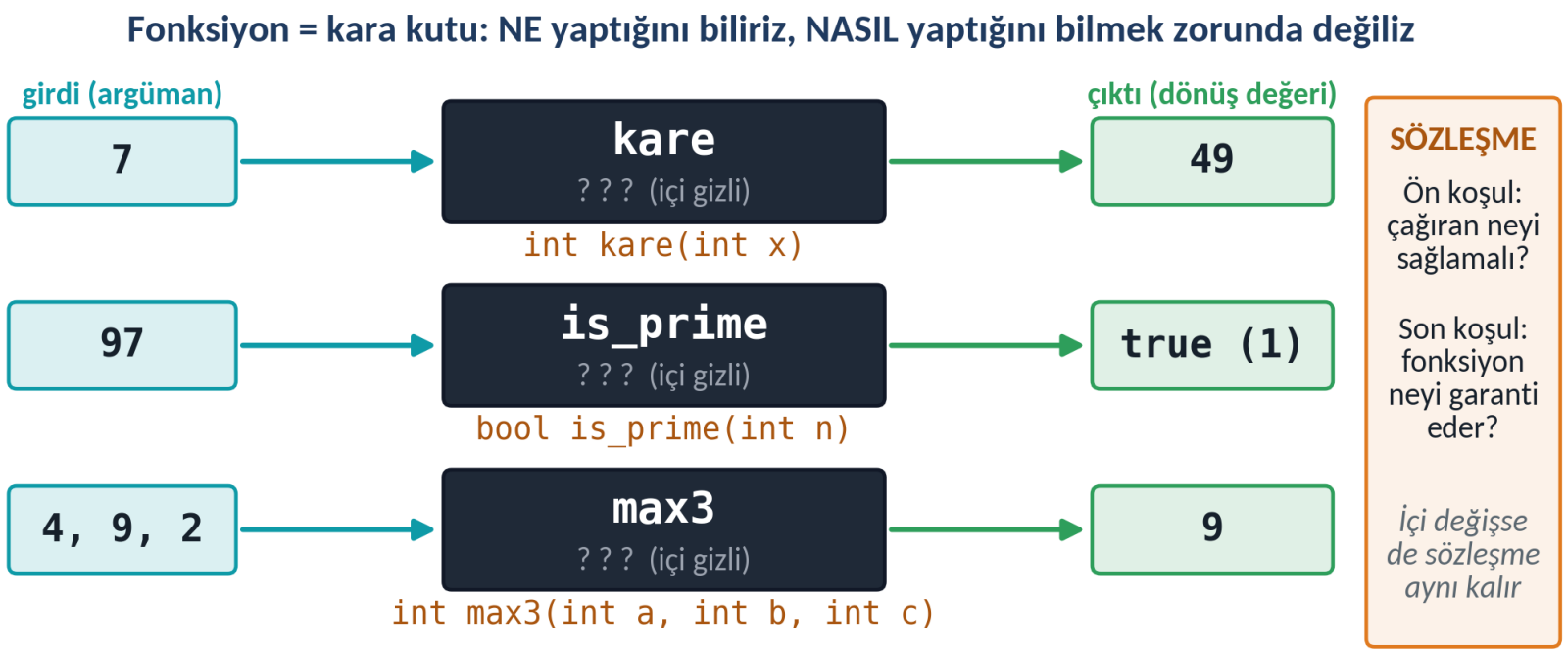
Yeniden kullanım: bir kez yaz, her yerden çağır

Okunabilirlik: `if (is_leap_year(yil))`

Test: parçayı programdan ayrı dene

KAVRAM

Fonksiyon = kara kutu



- Argümanlar **girer**, dönüş değeri **çıkar**
- İçi gizli: kullanan yalnızca **sözleşmeyi** bilir
- `is_prime(97)` → `true`: nasıl hesaplandığı önemsiz
- İç değişir, sözleşme aynı kalırsa çağıran etkilenmez

Fonksiyonun anatomisi

H08_02_kare.c

```c
int kare(int x);          ← prototip (bildirim)

int kare(int x)           ← tanım başlığı (; YOK)
{
    return x * x;
}
```

1. **DÖNÜŞ TİPİ**
   int: döndürülen değerin tipi; değer yoksa void
2. **FONKSİYON ADI**
   kare: anlamlı, fiil ya da soru (is_prime)
3. **PARAMETRE LİSTESİ**
   (int x): tip + ad; yoksa (void)
4. **GÖVDE { }**
   fonksiyonun yaptığı iş; yerel değişkenler
5. **return**
   değeri çağırana geri verir ve fonksiyonu bitirir

- **Dönüş tipi · ad · parametre listesi · gövde**
- `return ifade;` değeri verir ve **bitirir**
- Prototip = başlık + `;`
- Çağrı: `kare(a)` bir **ifadedir**

ÇAĞRI (main içinde)

```c
int sonuc = kare(a);
```

a: ARGÜMAN (çağıranın gönderdiği değer: 7)

x: PARAMETRE (fonksiyonun içindeki yeni değişken; a'nın kopyasıyla başlar)

`kare(7) → 49`

dönüş değeri, çağrı ifadesinin yerini alır: sonuc = 49

ÖRNEK 1 ★

İlk fonksiyon: selamla() — prototip, çağrı, tanım

H08_01_selamla.c · satır 10–19

```c
10  void selamla(void);                    /* prototip: "böyle bir fonksiyon" */
11
12  int main(void)
13  {
14      printf("main başladı.\n");
15      selamla();                         /* çağrı: akış selamla'ya gider... */
16      selamla();                         /* ...ve her seferinde buraya döner */
17      printf("main bitti.\n");
18      return 0;
19  }
```

H08_01_selamla.c · satır 21–26

```c
21  /* Ekrana iki satırlık bir karşılama iletisi yazar. */
22  void selamla(void)                     /* tanım: başlık + gövde */
23  {
24      printf("  Merhaba! AP-1'e hoş geldiniz.\n");
25      printf("  Bu hafta: Fonksiyonlar I\n");
26  }
```

GERÇEK ÇIKTI

```
main başladı.
  Merhaba! AP-1'e hoş geldiniz.
  Bu hafta: Fonksiyonlar I
  Merhaba! AP-1'e hoş geldiniz.
  Bu hafta: Fonksiyonlar I
main bitti.
```

Akış: `main` → `selamla` → `main` → `selamla` → `main`: her çağrı gövdeye **gider** ve çağrının arkasına **döner**.

ÖRNEK 2 ★

Parametre ve dönüş değeri: kare(x)

H08_02_kare.c · satır 10–21

```c
int kare(int x);                        /* prototip: int alır, int döndürür */

int main(void)
{
    int a = 7;
    int sonuc = kare(a);                /* argüman: a'nın DEĞERİ (7) */
    printf("kare(%d) = %d\n", a, sonuc);
    printf("kare(5) = %d\n", kare(5));
    printf("kare(a + 1) = %d\n", kare(a + 1));      /* önce a + 1 = 8 */
    printf("kare(kare(3)) = %d\n", kare(kare(3)));  /* içteki önce */
    return 0;
}
```

H08_02_kare.c · satır 23–27

```c
/* x'in karesini döndürür. */
int kare(int x)
{
    return x * x;
}
```

GERÇEK ÇIKTI

```
kare(7) = 49
kare(5) = 25
kare(a + 1) = 64
kare(kare(3)) = 81
```

- `return` değeri **verir** ve fonksiyonu **bitirir**
- `kare(a)` burada `49` gibi davranır
- Argüman: sabit, değişken, ifade ya da **başka bir çağrı**
- Sözleşme: $|x| \le 46\,340$

void'in iki anlamı ve () tuzağı

C17: () = "parametre bilgisi yok"

H08_K04_bos_parantez.c · satır 12–18

```c
12  int bugun();                              /* () : parametre bilgisi YOK */
13
14  int main(void)
15  {
16      printf("bugun(1, 2) = %d\n", bugun(1, 2));  /* yanlış, ama sessiz */
17      return 0;
18  }
```

GERÇEK: C17 SESSİZ, C23 HATA

```
$ gcc -std=c17 -Wall -Wextra -pedantic … && ./bos_parantez
bugun(1, 2) = 8
$ gcc -std=c2x …
…:16:34: error: too many arguments to function 'bugun'
```

Dönüş tipinde `void`: değer döndürmez

```
void cizgi(int uzunluk);
```

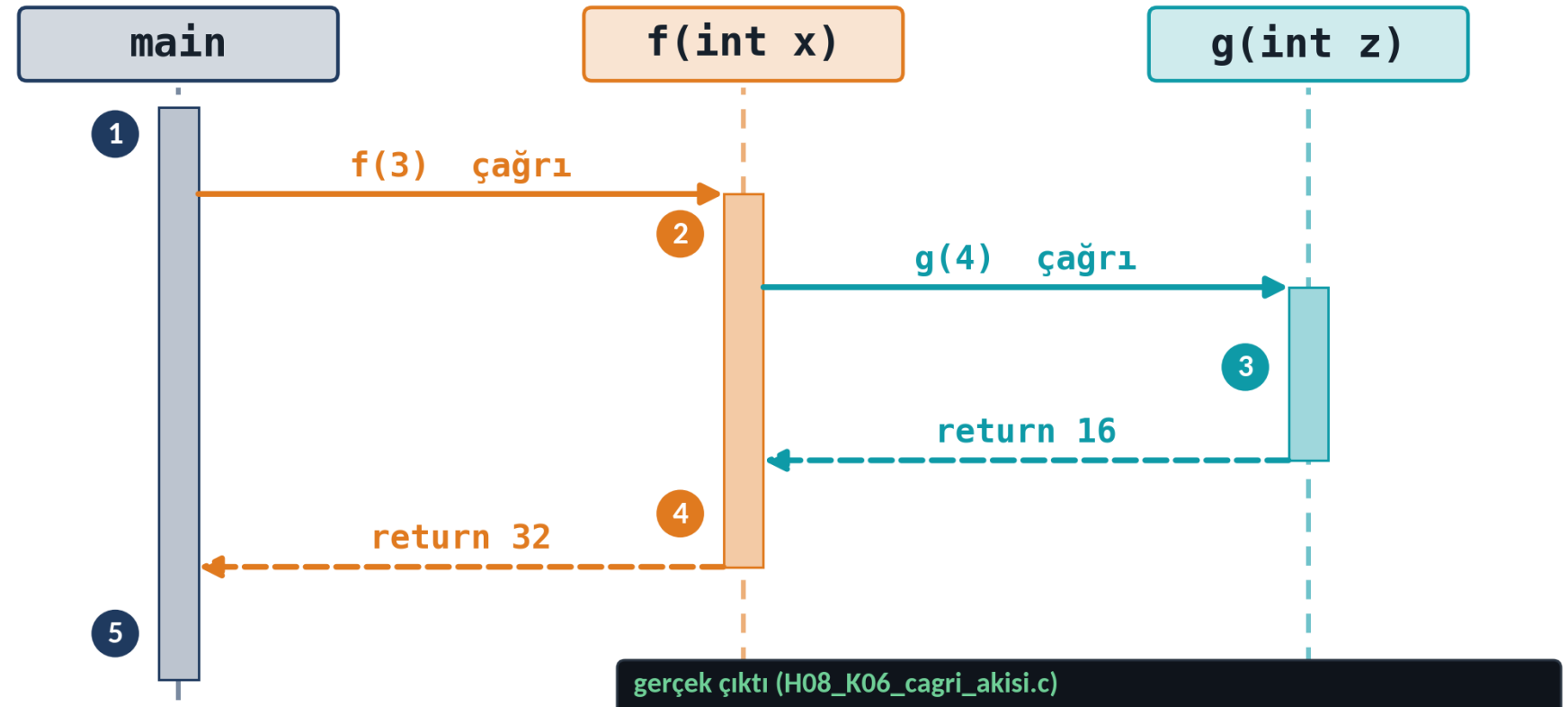
Parametre listesinde `(void)`: parametre almaz

```
int hafta_no(void);
```

Kural: parametresiz fonksiyonu her zaman `(void)` ile bildir: `int main(void)`

KAVRAM

Çağrı akışı: main → f → g → f → main



Her çağrı, çağrıldığı noktaya döner; dönüş değeri çağrı ifadesinin yerini alır: r = 32.

gerçek çıktı (H08_K06_cagri_akisi.c)

```
1. main: f(3) çağrılıyor
2.   f : x = 3, g(4) çağrılıyor
3.     g : z = 4, 16 döndürülüyor
4.   f : g'den 16 geldi, 32 döndürülüyor
5. main: r = 32, program bitiyor
```

- Her çağrı: **git** → çalış → **geri dön**
- Dönüş noktası: çağrının **hemen arkası**
- İç içe çağrıda en son giren **ilk çıkar**
- Kim nereye döneceğini nasıl hatırlar? → **yığın** (H9)

TAHMİN ET

Program ne yazdırır?

quiz/H08_Q11_deger_aktarim.c · satır 7–20

```c
void ikiye_katla(int x);

int main(void)
{
    int n = 5;
    ikiye_katla(n);
    printf("%d\n", n);
    return 0;
}

void ikiye_katla(int x)
{
    x = x * 2;
}
```

A `10`

B `5`

C `0`

D Derleme hatası: `void` fonksiyon `x`'i döndürmeli

KAVRAM

Değerle aktarım: iki ayrı bellek kutusu



- Argümanın **değeri** hesaplanır ve parametreye **kopyalanır**
- Parametre = fonksiyonun **yerel** değişkeni
- **Güvenlik:** çağıranı bozamaz
- **Sınır:** çağıranı değiştiremez → `return` (ya da işaretçi, H12)

02

Sözleşme, kütüphane, tasarım ve test

kara kutu ve sözleşme · bool ve bileşim · math, ctype, stdlib, time · adım adım iyileştirme · assert · kaputun altında

Sözleşme: ön koşul ve son koşul

H08_K02_kara_kutu.c · satır 24–35

```c
/* SÖZLEŞME (iki gerçekleştirim için de aynı)
 * Ön koşul : 0 <= n <= 100000
 * Son koşul: 1 + 2 + ... + n toplamını döndürür (n = 0 için 0).
 * Kara kutu 1: n tur süren döngü (H6, Örnek 2). */
long long toplam_dongu(int n)
{
    long long toplam = 0;
    for (int i = 1; i <= n; i++) {
        toplam += i;
    }
    return toplam;
}
```

H08_K02_kara_kutu.c · satır 37–41

```c
/* Kara kutu 2: sabit sayıda işlem, n(n+1)/2 (H2, Gauss). */
long long toplam_formul(int n)
{
    return (long long)n * (n + 1) / 2;
}
```

GERÇEK ÇIKTI: AYNI SÖZLEŞME

```
     n   toplam_dongu  toplam_formul
     0              0              0
    10             55             55
   100           5050           5050
100000     5000050000     5000050000
```

Ön koşul: çağıranın sağlaması gereken (0 ≤ n ≤ 100 000)

Son koşul: ön koşul sağlandıysa fonksiyonun garantisi

İki iç gerçekleştirim, **bir sözleşme**: çağıran için ikisi aynıdır

ÖRNEK 3–4

bool döndüren doğrulama ve bileşim

Örnek 4 ★ · bool döndüren soru

H08_04_cift_artik_yil.c · satır 35–40

```
35  /* Gregoryen kural: 4'e bölünen yıllar artıktır; ama 100'e bölünenler
36   * değildir; ancak 400'e bölünenler yine artıktır. */
37  bool is_leap_year(int yil)
38  {
39      return (yil % 4 == 0 && yil % 100 != 0) || yil % 400 == 0;
40  }
```

GERÇEK ÇIKTI · YİL_RAPORU (KESİT)

```
1900: artık yıl değil (365 gün)
2000: artık yıl (366 gün)
2024: artık yıl (366 gün)
2026: artık yıl değil (365 gün)
2100: artık yıl değil (365 gün)
```

`yil_raporu` **yazar**, `is_leap_year` **karar verir**: hesap ve çıktı ayrı.

Örnek 3 ★ · bileşim

H08_03_max2_max3.c · satır 24–30

```
24  int max2(int a, int b)
25  {
26      if (a >= b) {
27          return a;
28      }
29      return b;
30  }
```

H08_03_max2_max3.c · satır 33–36

```
33  int max3(int a, int b, int c)
34  {
35      return max2(max2(a, b), c);
36  }
```

GERÇEK ÇIKTI (KESİT)

```
max3(4, 9, 2)   = 9
max3(7, 7, 7)   = 7
```

Standart kütüphane: belgeden okuyarak kullanmak

<math.h>

```
sqrt(x) · pow(x, y)
fabs(x)
floor(x) · ceil(x)
```

double alır/döndürür; Linux: -lm

<ctype.h>

```
isdigit · isalpha
isspace · isupper
toupper · tolower
```

argüman: unsigned char ya da EOF

<stdlib.h>

```
abs(n) · exit(durum)
rand() · srand(t)
EXIT_SUCCESS
EXIT_FAILURE
```

genel amaçlı araçlar

<time.h>

```
time(NULL)
clock()
CLOCKS_PER_SEC (H7)
```

takvim zamanı, işlemci süresi

<assert.h>

```
assert(koşul)
NDEBUG → kapalı
```

geliştirme sırasında denetim

<stdbool.h>

```
bool
true · false
```

C99: mantıksal tip

- Her fonksiyon için 4 bilgi: **başlık**, **imza**, **anlam**, **özel durumlar**
- Linux'ta: `man 3 sqrt`
- Prototipler başlıkta; tanımlar **libc / libm**'de
- Dönüş değerini **kullanın**

<math.h> ve -lm: sabitle yapılan deneme yanıltır

H08_K08_math.c · satır 22–27

```c
22      printf("sqrt(%.2f)   = %.4f\n", x, sqrt(x));
23      printf("pow(%.2f, 3) = %.4f\n", x, pow(x, 3));
24      printf("fabs(%.2f)   = %.4f\n", x, fabs(x));
25      printf("floor(%.2f)  = %.4f\n", x, floor(x));
26      printf("ceil(%.2f)   = %.4f\n", x, ceil(x));
27      printf("hipotenus(3, 4) = %.4f\n", hipotenus(3, 4));
```

H08_K08_math.c · satır 32–35

```c
32  double hipotenus(double a, double b)
33  {
34      return sqrt(a * a + b * b);
35  }
```

GERÇEK ÇIKTI · GİRDİ −2.5

```
$ ./math
Bir sayı girin: -2.5
sqrt(-2.50)   = -nan
pow(-2.50, 3) = -15.6250
fabs(-2.50)   = 2.5000
floor(-2.50)  = -3.0000
ceil(-2.50)   = -2.0000
hipotenus(3, 4) = 5.0000
```

GERÇEK: -LM UNUTULUNCA (H08_K10: SQRT(X), DEĞİŞKEN ARGÜMAN)

```
$ gcc -std=c17 -Wall -Wextra -pedantic -g H08_K10_lm_degisken.c -o lm_degisken_lmsiz
H08_K10_lm_degisken.c:18:(.text+0x60): undefined reference to `sqrt'
collect2: error: ld returned 1 exit status
$ gcc -std=c17 -O0 -S H08_K09_lm_sabit.c -o - | grep -cE 'call\s+(sqrt|pow)'
0        # H08_K09: sabit argüman, derleyici önceden hesapladı → -lm gerekmedi
```

<ctype.h>: karakter sınıfları

H08_K11_ctype.c · satır 23–32

```c
    while ((c = getchar()) != '\n' && c != EOF) {
        if (isdigit(c)) {
            rakam++;
        } else if (isalpha(c)) {
            harf++;
        } else if (isspace(c)) {
            bosluk++;
        }
        putchar(toupper(c));            /* harf değilse aynen döner */
    }
```

- `isdigit('7')` = **2048**: "doğru" = sıfırdan farklı herhangi bir int
- Sonucu `== 1` ile **karşılaştırmayın**
- UTF-8'de Ç, ğ, ı iki bayt: ctype onları harf saymaz (H13)

H08_K11_ctype.c · satır 38–41

```c
int rakam_mi(char ch)
{
    return isdigit((unsigned char)ch) != 0;
}
```

`char` → önce `(unsigned char)`: negatif değer UB

GERÇEK ÇIKTI · GİRDİ "ÇAĞRI 2026"

```
$ ./ctype_ornek
isdigit('7') = 2048, isdigit('x') = 0
rakam_mi('5') = 1
Bir satır yazın: Çağrı 2026
ÇAğRı 2026
rakam: 4, harf: 2, boşluk: 1
```

<stdlib.h> ve <time.h>: exit, rand, time

H08_K12_stdlib.c · satır 26–34

```c
/* Ön koşul: b != 0. İhlal edilirse programı EXIT_FAILURE ile bitirir. */
int bol(int a, int b)
{
    if (b == 0) {
        printf("Hata: sıfıra bölme! Program sonlandırılıyor.\n");
        exit(EXIT_FAILURE);            /* main'e dönmeden biter */
    }
    return a / b;
}
```

- `exit` programı **bulunduğu yerden** bitirir
- `EXIT_SUCCESS` / `EXIT_FAILURE`: taşınabilir kodlar
- `time(NULL)` = 1790836264: 1.1.1970'ten beri saniye

GERÇEK ÇIKTI · B = 0

```
$ ./stdlib_ornek
abs(-7) = 7, abs(7) = 7
İki tamsayı: 17 0
Hata: sıfıra bölme! Program sonlandırılıyor.
[çıkış kodu: 1]
```

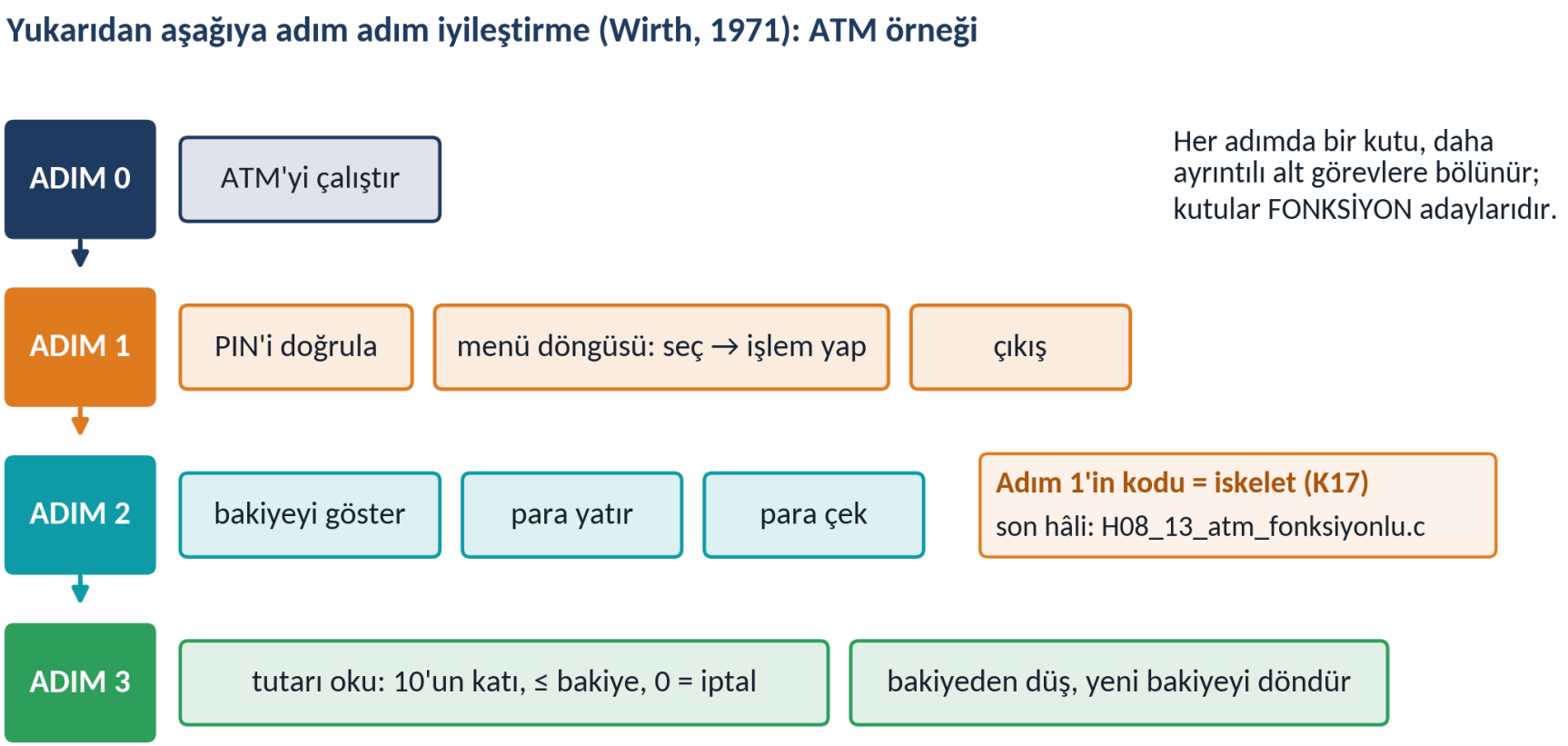
H08_K14_rand_aralik.c · satır 36–39

```c
int rastgele(int alt, int ust)
{
    return alt + rand() % (ust - alt + 1);
}
```

GERÇEK ÇIKTI · -DSABIT_TOHUM=2026

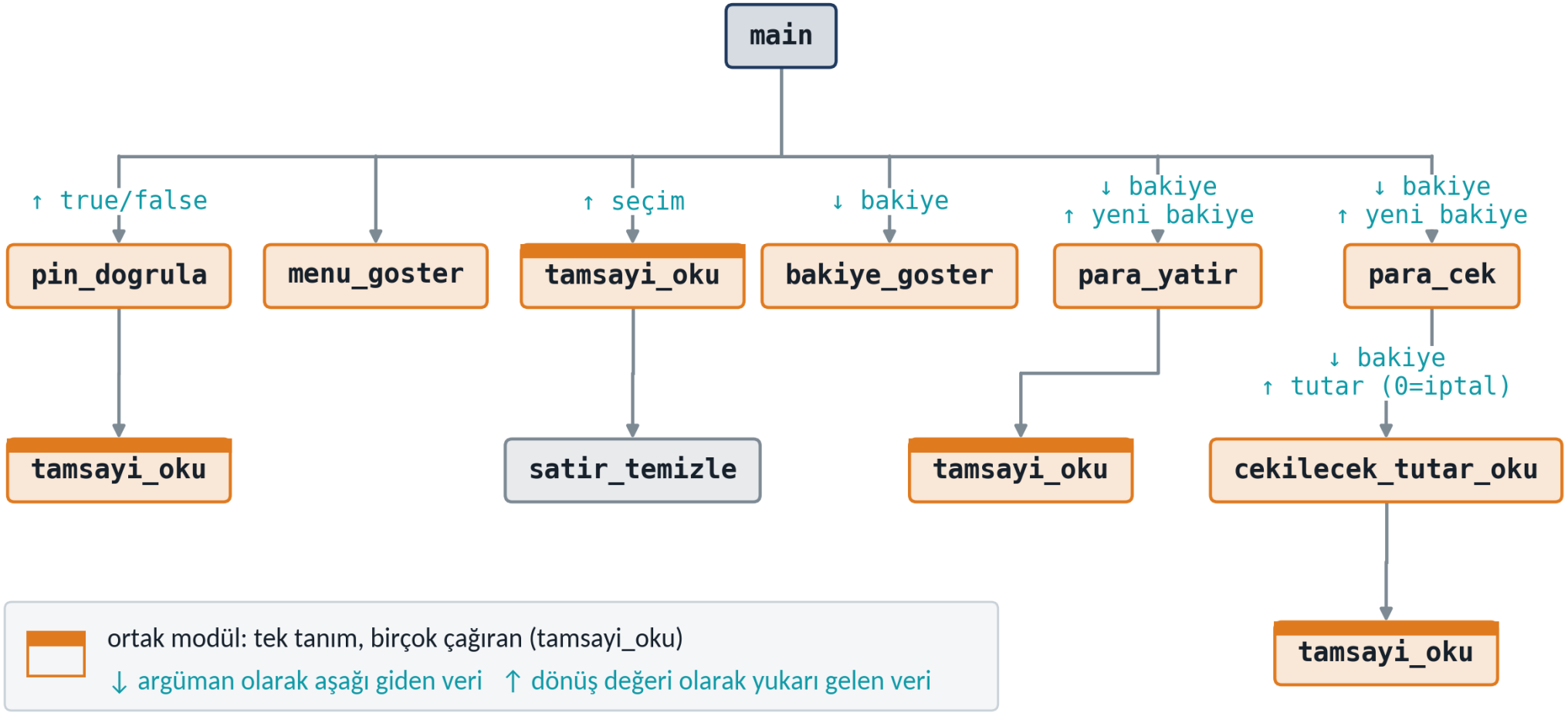
```
Zar       : 2 1 2 4 5 6 6 3 5 4
Not 0-100: 11 71 57 93 63
```

Yukarıdan aşağıya: adım adım iyileştirme

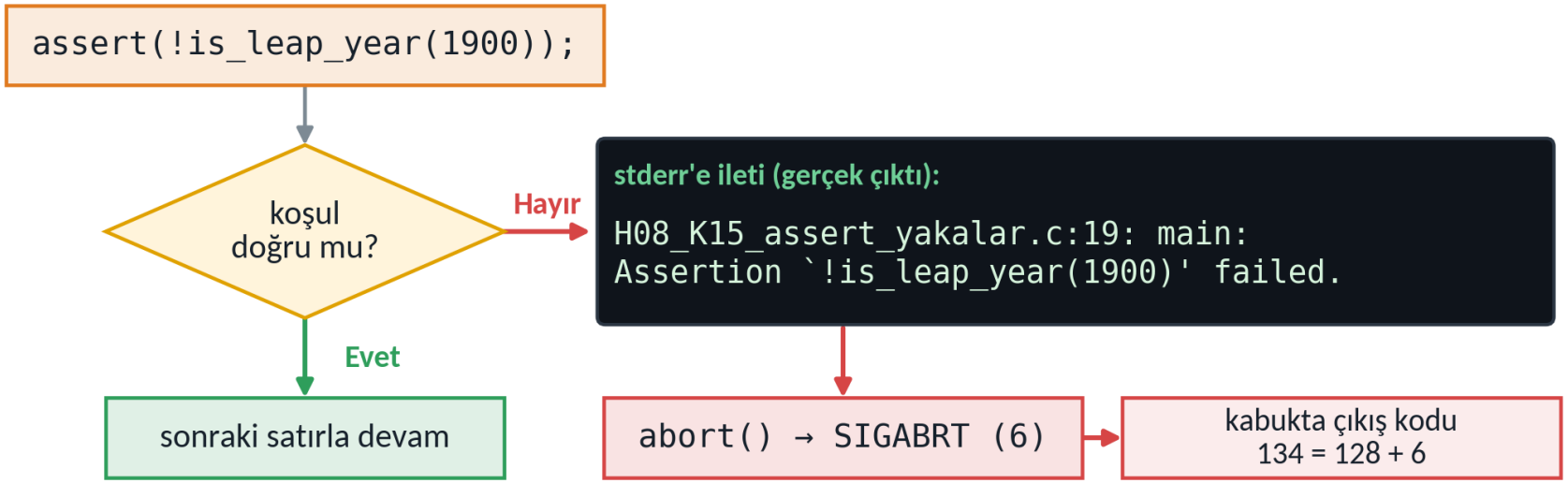


- Problemi **tek cümleyle** yaz
- Her adımda bir kutuyu **alt görevlere** ayır
- Koda dökülecek kadar basitleşince **dur**
- Her kutu bir **fonksiyon adayı**
- İlk adım: **koçan** (stub) fonksiyonlarla derlenen iskelet

Yapı şeması: kim kimi çağırır?



assert ile birim testi



-DNDEBUG ile derlenince:

assert satırları önişlemcide SİLİNİR; koşul hiç denetlenmez (yan etkili ifadeyi assert içine yazmayın).

- `assert(koşul)`: doğruysa **sessiz**
- Yanlışsa: dosya, satır, koşul → `abort()`
- `-DNDEBUG`: assert'ler **silinir**
- Programcı hataları için; **kullanıcı girdisi için değil**

assert hatayı yakalar; NDEBUG onu gizler

H08_K15_assert_yakalar.c · satır 15–23

```c
15  int main(void)
16  {
17      assert(is_leap_year(2024));
18      printf("1. test geçti: 2024 artık yıl\n");
19      assert(!is_leap_year(1900));
20      printf("2. test geçti: 1900 artık yıl değil\n");
21      printf("Tüm testler geçti.\n");
22      return 0;
23  }
```

HATALI gerçekleştirim: 100/400 kuralı yok

H08_K15_assert_yakalar.c · satır 26–29

```c
26  bool is_leap_year(int yil)
27  {
28      return yil % 4 == 0;
29  }
```

- `abort()` → SIGABRT (6) → kabuk **134** = 128 + 6

GERÇEK ÇIKTILAR

```
$ ./assert_yakalar; echo "[çıkış kodu: $?]"
1. test geçti: 2024 artık yıl
assert_yakalar: H08_K15_assert_yakalar.c:19: main: Assertion `!is_leap_year(1900)' failed.
bash: line 1:  9232 Aborted                 ./assert_yakalar
[çıkış kodu: 134]
$ ./assert_yakalar_ndebug        # aynı kaynak, -DNDEBUG ile derlendi
Tüm testler geçti.
[çıkış kodu: 0]
```

KAPUTUN ALTINDA

Bir çağrıda neler olur? call, ret ve yazmaçlar

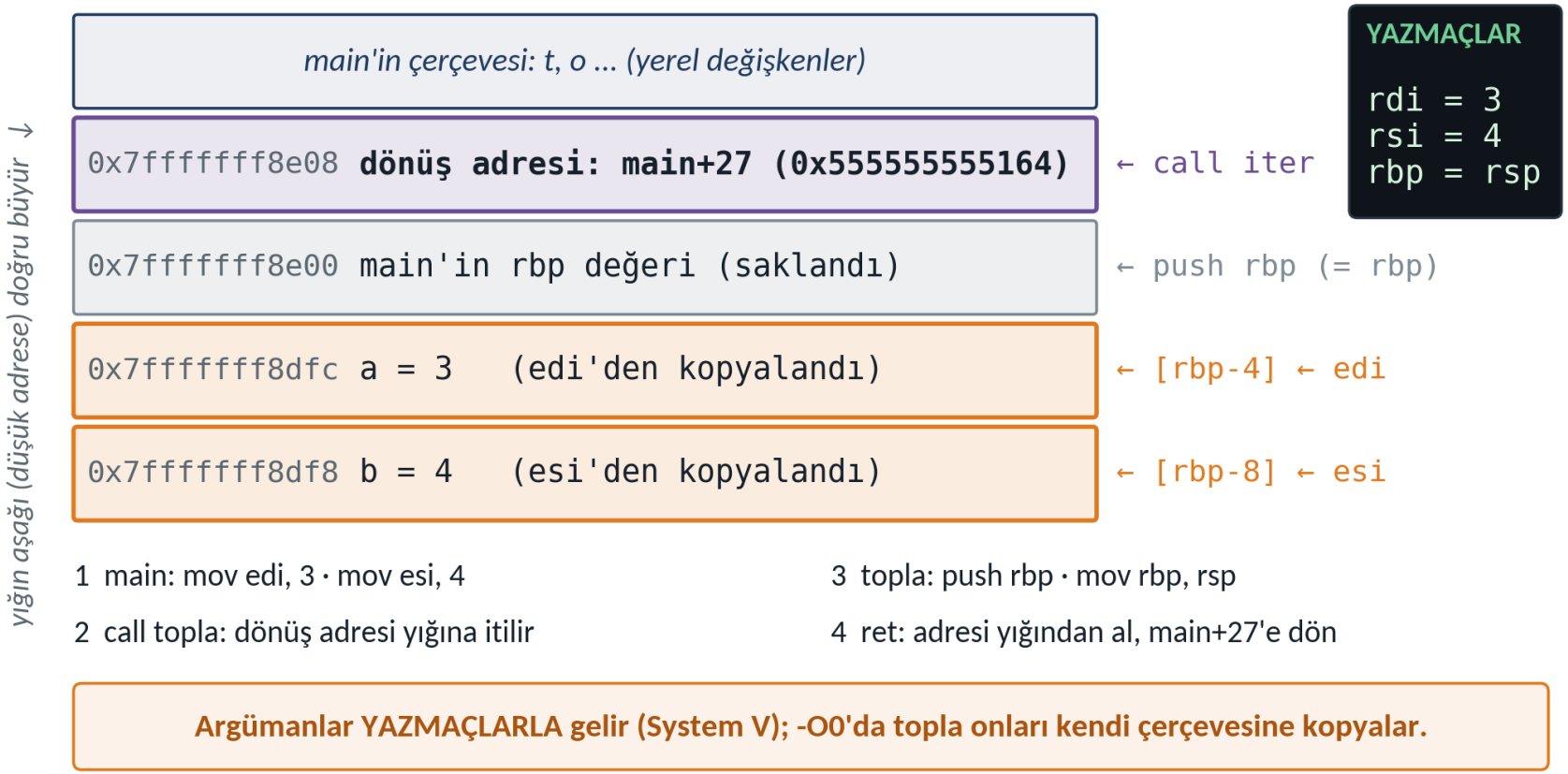
- **1.** Argümanlar çağrılanın bulacağı yere konur: x86-64 Linux'ta (System V ABI) ilk tamsayılar **yazmaçlarda**: `edi`, `esi` …
- **2.** `call`: **dönüş adresini** yığına iter ve fonksiyona atlar
- **3.** Çağrılan **çerçevesini** kurar: `push rbp`, parametreleri kendi çerçevesine **kopyalar**
- **4.** Dönüş değeri `eax`'te; `ret` adresi yığından alıp **geri döner**
- "Argümanlar yığına itilir" bu platformda **yanlış** bir genellemedir

GERÇEK GCC -O0 -S ÇIKTISI (KESİT)

```
$ gcc -O0 -S -masm=intel … H08_K07_asm_cagri.c
main:
    ...
    mov  esi, 4
    mov  edi, 3
    call  topla
    ...
topla:
    endbr64
    push  rbp
    mov  rbp, rsp
    mov  DWORD PTR [rbp-4], edi
    mov  DWORD PTR [rbp-8], esi
    mov  edx, DWORD PTR [rbp-4]
    mov  eax, DWORD PTR [rbp-8]
    add  eax, edx
    pop  rbp
    ret
```

gdb ile yığın çerçevesi: topla(3, 4)

topla(3, 4) çağrısında yığın: topla'ya girildiği an (gdb, H08_K07_asm_cagri.c)



1 main: mov edi, 3 · mov esi, 4

2 call topla: dönüş adresi yığına itilir

3 topla: push rbp · mov rbp, rsp

4 ret: adresi yığından al, main+27'e dön

Argümanlar YAZMAÇLARLA gelir (System V); -O0'da topla onları kendi çerçevesine kopyalar.

- `rdi` = 3, `rsi` = 4: argümanlar **yazmaçta**
- `rbp + 8`: dönüş adresi `main+27`
- `objdump`: `call topla`'dan **sonraki** komut
- `&a` = 0x7fffffff8dfc = `rbp − 4`

Çağrı kuralı platforma bağlıdır: System V ve Windows x64

	System V: x86-64 Linux/macOS (Intel)	Windows x64 (MSVC, MinGW)
1.–4. tamsayı arg.	`rdi, rsi, rdx, rcx`	`rcx, rdx, r8, r9`
5.–6. tamsayı arg.	`r8, r9 (sonrası yığında)`	`yığında`
kayan noktalı arg.	`xmm0 – xmm7`	`xmm0 – xmm3 (konuma göre)`
dönüş değeri	`rax / eax · double: xmm0`	`rax / eax · double: xmm0`
çağıranın ayırdığı alan	–	`32 bayt gölge alan`

topla(3, 4) çağrısı · gerçek gcc -O0 -S çıktısı:

edi = rdi'nin alt 32 biti; int için yeterli.

```
gcc -O0 -S -masm=intel

mov esi, 4
mov edi, 3
call topla
```

```
aynı kaynak, -mabi=ms

sub rsp, 48
mov edx, 4
mov ecx, 3
call topla
```

- C kaynağı **aynı**; farkı derleyici halleder
- Windows x64: `rcx`, `rdx`, `r8`, `r9` + 32 bayt **gölge alan**
- `-O2`: `topla` **satır içine** alındı (`mov edx, 7`)
- Çerçeveleri **Python Tutor** (C) ile izleyin

03

Örnek merdiveni

13 basamak · H6–H7 kodlarını fonksiyonlara taşımak · amiral örnekler: is_prime ve ATM

H6 döngüleri fonksiyon gövdesi olur

Örnek 5 ★★ · faktoriyel (H06_05)

H08_05_faktoriyel_us.c · satır 29–36

```
29  unsigned long long faktoriyel(int n)
30  {
31      unsigned long long carpim = 1;      /* çarpım biriktirici (H6) */
32      for (int i = 2; i <= n; i++) {
33          carpim *= i;
34      }
35      return carpim;
36  }
```

GERÇEK ÇIKTI (KESİT)

```
0!  = 1
20! = 2432902008176640000
2^10   = 1024
C(5, 2) = 5! / (2! * 3!) = 10
```

Örnek 7 ★★ · ekok, ebob'u kullanır (H06_11)

H08_07_ebob_ekok.c · satır 39–45

```
39  /* Ön koşul : a > 0, b > 0.
40   * Son koşul: a ile b'nin en küçük ortak katı.
41   * ebob'u KULLANIR; önce böl, sonra çarp: ara sonuç küçük kalır. */
42  long long ekok(int a, int b)
43  {
44      return (long long)a / ebob(a, b) * b;
45  }
```

GERÇEK ÇIKTI · ÖKLİD ÖRNEĞİ

```
$ ./ebob_ekok
Pozitif iki tamsayı girin: 1071 462
EBOB(1071, 462) = 21
EKOK(1071, 462) = 23562
```

Amiral örnek · Örnek 6 ★★: is_prime(n)

Problem. H7'deki asallık testi `main`'in içindeydi (`H07_06_asal_naif.c`, `H07_11_asal_karekok_sure.c`). Hızlı sürümü her yerden çağrılabilen, test edilebilen `bool is_prime(int n)`'e dönüştür; davranış değişmesin (**yeniden düzenleme**).

GİRDİ

- n: herhangi bir int
- negatif, 0, 1 dahil

→

İŞLEM

- n < 2 → false; n = 2 → true
- çiftse → false
- d = 3, 5, 7 … ve d ≤ n / d:
- n % d = 0 → false
- bölen yoksa → true

→

ÇIKTI

- bool: asalsa true
- ekrana hiçbir şey yazmaz

Kısıtlar: ön koşul yok, **her** `int` için yanıt · 2 tek çift asal · 2 147 483 647 ($2^{31} - 1$) asal ve döngü koşulu orada **taşmamalı** → `d * d <= n` yerine `d <= n / d`

Örnek 6: kod ve satır satır

H08_06_is_prime.c · satır 37–47

```
37  bool is_prime(int n)
38  {
39      if (n < 2) {
40          return false;                   /* 0, 1 ve negatifler asal değil */
41      }
42      if (n == 2) {
43          return true;                    /* tek çift asal */
44      }
45      if (n % 2 == 0) {
46          return false;                   /* diğer bütün çiftler */
47      }
```

H08_06_is_prime.c · satır 48–54

```
48      for (int d = 3; d <= n / d; d += 2) {
49          if (n % d == 0) {
50              return false;               /* bölen bulundu: hemen dön */
51          }
52      }
53      return true;                        /* hiçbir tek bölen yok */
54  }
```

GERÇEK ÇIKTI (KESİT)

```
(15 asal)
is_prime(97)          = 1
is_prime(91)          = 0
is_prime(-7)          = 0
is_prime(2147483647) = 1
```

- Birden çok `return`: sınır durumları **erken** elenir
- `d <= n / d`: anlamı `d * d <= n` ile aynı, ama **taşmaz**
- Bölen bulununca `return false`: H7'deki bayrak ve `break` gerekmez

Örnek 6: izleme tablosu (91 ve 97)

H08_06_is_prime.c · satır 48–53

```
48      for (int d = 3; d <= n / d; d += 2) {
49          if (n % d == 0) {
50              return false;              /* bölen bulundu: hemen dön */
51          }
52      }
53      return true;                       /* hiçbir tek bölen yok */
```

n / d **tamsayı** bölmesidir: 97 / 11 = 8

Çağrı	d	koşul d ≤ n / d	n % d	sonuç
is_prime(91)	3	3 ≤ 30 → doğru	1	d += 2
is_prime(91)	5	5 ≤ 18 → doğru	1	d += 2
is_prime(91)	**7**	**7 ≤ 13 → doğru**	**0**	**return false**
is_prime(97)	3	3 ≤ 32 → doğru	1	d += 2
is_prime(97)	5	5 ≤ 19 → doğru	2	d += 2
is_prime(97)	7	7 ≤ 13 → doğru	6	d += 2
is_prime(97)	9	9 ≤ 10 → doğru	7	d += 2
is_prime(97)	**11**	**11 ≤ 8 → yanlış**	**—**	**return true**

Örnek 6: test tablosu ve taşma kanıtı

Test	Girdi	Beklenen	Gerçek	Sonuç
Normal (asal)	97	1	1	✓
Asal görünümlü	91 = 7 · 13	0	0	✓
Sınır: liste	1..50	15 asal, 1 yok	15 asal; ilki 2	✓
Sınır: INT_MAX	2147483647	1, taşma yok	1 (UBSan temiz)	✓
Hatalı girdi	-7	0	0	✓

GERÇEK ÇIKTI (DOSYA ADI KISALTILDI)

```
$ gcc … -fsanitize=undefined H08_K18_karekok_tasma.c && ./karekok_tasma     # d * d <= n sürümü
is_prime_dd(97) = 1
…:32:23: runtime error: signed integer overflow: 46341 * 46341 cannot be represented in type 'int'
is_prime_dd(2147483647) = 0
```

`d * d <= n` sürümü $2^{31} - 1$ için **yanlış** yanıt verdi: tanımsız davranış her zaman çökme değildir.

Örnek 8–9: menü + fonksiyonlar, küçük fonksiyonlardan büyük çözüm

Örnek 8 ★★ · birim dönüştürücü

H08_08_birim_donusturucu.c · satır 28–32

```
28          switch (secim) {
29          case 1:
30              x = deger_oku();
31              printf("%.2f °C = %.2f °F\n", x, c_to_f(x));
32              break;
```

GERÇEK ÇIKTI · KISA OTURUM

```
$ ./birim

1) °C->°F  2) °F->°C  3) km->mil  4) mil->km  0) Çıkış
Seçim: 1
Değer: 100
100.00 °C = 212.00 °F

1) °C->°F  2) °F->°C  3) km->mil  4) mil->km  0) Çıkış
Seçim: 0
Güle güle.
[çıkış kodu: 0]
```

Örnek 9 ★★ · palindrom

H08_09_basamak_palindrom.c · satır 44–52

```
44  long long ters_cevir(int n)
45  {
46      long long ters = 0;
47      while (n > 0) {
48          ters = ters * 10 + n % 10;
49          n /= 10;
50      }
51      return ters;
52  }
```

H08_09_basamak_palindrom.c · satır 55–58

```
55  bool is_palindrome(int n)
56  {
57      return ters_cevir(n) == n;
58  }
```

12321 → **evet** · 1230 → tersi **321** · 1999999999 → **9999999991** (long long)

Örnek 10 ★★: “Neden çalışmıyor?” takas(a, b)

H08_10_takas_denemesi.c · satır 17–20

```c
17      int a = 3, b = 7;
18      printf("main  (önce) : a = %d, b = %d   &a = %p\n", a, b, (void *)&a);
19      takas(a, b);
20      printf("main  (sonra): a = %d, b = %d   <- değişmedi!\n", a, b);
```

H08_10_takas_denemesi.c · satır 24–32

```c
24  /* AMAÇ: a ile b'yi değiş tokuş etmek. Neden işe yaramıyor? */
25  void takas(int a, int b)
26  {
27      printf("takas (giriş): a = %d, b = %d   &a = %p\n", a, b, (void *)&a);
28      int gecici = a;                    /* H3'teki geçici değişkenli takas */
29      a = b;
30      b = gecici;
31      printf("takas (çıkış): a = %d, b = %d\n", a, b);
32  }
```

GERÇEK ÇIKTI · 1. ÇALIŞTIRMA (ADRESLER HER ÇALIŞTIRMADA DEĞİŞİR)

```
main  (önce) : a = 3, b = 7   &a = 0x7ffd05ff2fc0
takas (giriş): a = 3, b = 7   &a = 0x7ffd05ff2f9c
takas (çıkış): a = 7, b = 3
main  (sonra): a = 3, b = 7   <- değişmedi!
```

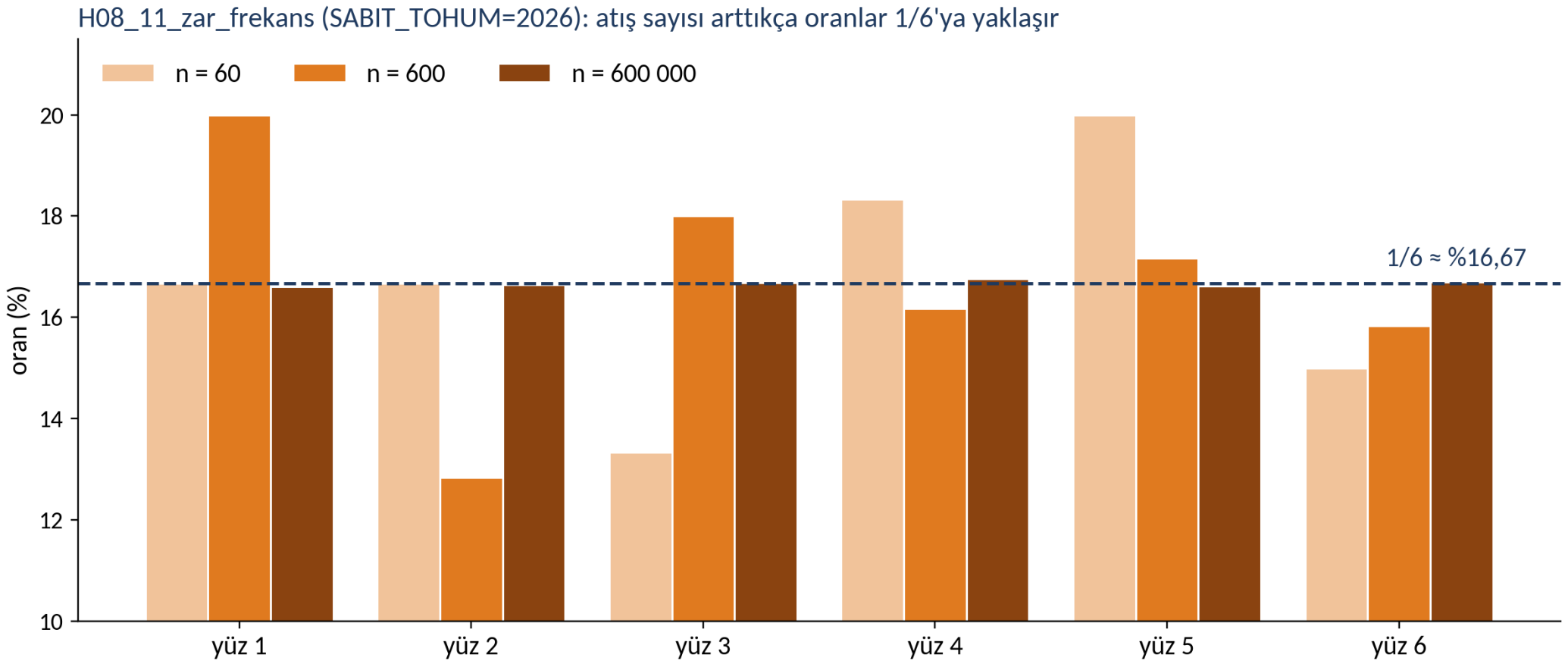
takas(a, b) neden çalışmıyor: bellekte ne oluyor?



İki a, iki farklı adreste iki ayrı değişkendir. Çözüm H12'de: adresleri (işaretçi) göndermek.

- `takas`'a **değerler** gitti; kopyalar takas edildi
- `main`'in `a`'sı 0x7ffd05ff2fc0; `takas`'ınki 0x7ffd05ff2f9c
- Çerçeve silinince takas da **kayboldu**
- Çözüm: **adres** göndermek → işaretçi (H12)

Örnek 11 ★★: zar simülasyonu ve büyük sayılar yasası



switch (zar_at()) + altı sayaç (s1 … s6): dizi yok! · 60 atış: %13,33–%20,00 · 600 000 atış: %16,60–%16,76

Örnek 12 ★★★: assert ile is_prime ve ebob testleri

H08_12_assert_testleri.c · satır 36–50

```c
int test_is_prime(void)
{
    assert(is_prime(7));                /* normal */
    assert(is_prime(97));
    assert(!is_prime(91));              /* 7 * 13: asal "görünümlü" */
    assert(!is_prime(0));               /* sınır */
    assert(!is_prime(1));
    assert(is_prime(2));
    assert(is_prime(3));
    assert(!is_prime(4));
    assert(is_prime(2147483647));       /* INT_MAX: taşma olmamalı */
    assert(!is_prime(-7));              /* beklenmedik: negatif */
    printf("is_prime   : 10 test geçti\n");
    return 10;
}
```

GERÇEK ÇIKTI (-LM İLE)

```
$ ./assert_testleri
is_prime   : 10 test geçti
ebob/ekok  : 8 test geçti
ortalama   : 6 test geçti
Tüm testler geçti (24 doğrulama).
```

KAYAN NOKTA: == YERİNE YAKİN()

```
$ ./kayan_tolerans
0.1 + 0.2     = 0.30000000000000004
a == 0.3      : 0
yakin(a, 0.3) : 1
```

- Testler: **normal · sınır · hatalı** girdi
- Her test fonksiyonu doğrulama sayısını döndürür

Amiral örnek · Örnek 13 ★★★: ATM'yi fonksiyonlara bölmek

H7 sürümünde `scanf` + EOF + tampon temizleme bloğu **4 kez** yazılmıştı:

PIN	geçersiz: −1
menü seçimi	geçersiz: 0
yatırılacak tutar	geçersiz: 0
çekilecek tutar	geçersiz: −1

↓

```
int tamsayi_oku(int gecersiz)
```

Fark **parametre** oldu: 4 kopya → 1 fonksiyon

H08_13_atm_fonksiyonlu.c · satır 84–97

```
84  int tamsayi_oku(int gecersiz)
85  {
86      int deger;
87      int okunan = scanf("%d", &deger);
88      if (okunan == EOF) {
89          printf("\nGirdi sona erdi.\n");
90          exit(EXIT_FAILURE);             /* EXIT_FAILURE = 1 */
91      }
92      if (okunan == 0) {
93          satir_temizle();
94          return gecersiz;
95      }
96      return deger;
97  }
```

Örnek 13: main yalnızca ne yapılacağını söyler

H08_13_atm_fonksiyonlu.c · satır 40–44

```c
40      printf("===== AP-1 BANKASI ATM =====\n");
41      if (!pin_dogrula()) {
42          printf("Kartınız bloke edildi. Lütfen şubenize başvurun.\n");
43          return 2;
44      }
```

H08_13_atm_fonksiyonlu.c · satır 47–58

```c
47      do {
48          menu_goster();
49          secim = tamsayi_oku(0);          /* sayı değilse 0: geçersiz seçim */
50          switch (secim) {
51          case 1:
52              bakiye_goster(bakiye);
53              break;
54          case 2:
55              bakiye = para_yatir(bakiye);    /* kopya gider, yenisi döner */
56              break;
57          case 3:
58              bakiye = para_cek(bakiye);
```

- `main` **ne** yapılacağını söyler, **nasıl**ını değil
- `pin_dogrula()` → `bool`: koşul gibi okunur
- Bakiye `main`'in yerel değişkeni: fonksiyona **kopyası** gider, yenisi **döner**
- Atama unutulursa yatırılan para **kaybolur** (HA3)
- Kurallar: dosya kapsamındaki sabitler (H9)

Örnek 13: bakiye çağrılar arasında nasıl dolaşır? Davranış aynı mı?

İzleme · girdi `1111 1234 1 3 250 2 100 4` (gerçek çıktıyla doğrulandı)

Adım	Çağrı	Parametre (kopya)	Dönüş değeri	main'deki bakiye
1	`pin_dogrula()`	–	true	1000
2	`bakiye_goster(bakiye)`	`bakiye = 1000`	(void)	1000
3	`para_cek(bakiye)` çağrılır	`bakiye = 1000`	(henüz dönmedi)	1000
3a	`cekilecek_tutar_oku(bakiye)`	`bakiye = 1000`	250	1000
3	**`para_cek` döner**	**–**	**750**	**750**
4	**`para_yatir(bakiye)`**	**`bakiye = 750`**	**850**	**850**
5	`tamsayi_oku(0)` → 4	`gecersiz = 0`	4 (çıkış)	850

Regresyon testi · aynı girdiler `H07_14_atm_menu.c` ve bu programla; çıktılar `diff` ile karşılaştırıldı

Senaryo	Girdi	Çıkış kodu (H7 / H8)	diff
başlıktaki örnek: bir hatalı PIN, bakiye, çekme, yatırma	1111 1234 1 3 250 2 100 4	0 / 0	fark yok
geçersiz girdiler	abc 1234 x 9 2 0 abc 20000 500 3 1 …	0 / 0	fark yok
üç hatalı PIN: bloke	1 2 abc	2 / 2	fark yok
menüde girdi biter (EOF)	1234 1	1 / 1	fark yok

Hata Avcısı: altı fonksiyon hatası

#	Hata	Derleyici / belirti (gerçek)	Çözüm
HA1	Prototip eksik	GCC 13: uyarı (implicit declaration); Clang 18, GCC 14: hata	Prototipi çağrıdan önce yaz
HA2	return unutulmuş	uyarı: control reaches end of non-void function; sonuç rastgele	Her yol bir return'e ulaşsın
HA3	Dönüş değeri kullanılmamış	kendi fonksiyonumuz için uyarı yok; sonuç kaybolur	Sonucu ata: x = f(x);
HA4	Parametre sayısı / tipi	sayı: hata (too few arguments); tip: sessiz dönüşüm	İmzaya uy; -Wconversion
HA5	Tanım başlığına ;	hata: expected identifier or '(' before '{' token	Tanımda ; yok
HA6	Fonksiyon içinde fonksiyon	GCC: -pedantic uyarısı (uzantı); Clang: hata	Fonksiyonları yan yana yaz

HA1 · prototip eksik ve HA2 · return unutulmuş

HA1 · prototip yok

hata_avcisi/H08_HA1_prototip_eksik.c · satır 11–20

```
11  int main(void)
12  {
13      printf("kup(3) = %d\n", kup(3));
14      return 0;
15  }
16
17  int kup(int x)
18  {
19      return x * x * x;
20  }
```

GERÇEK İLETİLER (KISALTILMIŞ)

```
$ gcc -std=c17 -Wall … (GCC 13.3)
13:29: warning: implicit declaration of function ‘kup’
$ ./ha1
kup(3) = 27       # şans eseri doğru
$ clang -std=c17 … (Clang 18)
13:29: error: call to undeclared function 'kup'
```

HA2 · bir yol return’süz

hata_avcisi/H08_HA2_return_unutma.c · satır 19–24

```
19  int max2(int a, int b)
20  {
21      if (a > b) {
22          return a;
23      }
24  }
```

GERÇEK: MAX2(4, 9) HİÇBİRİNDE 9 DEĞİL

```
warning: control reaches end of non-void function
max2(4, 9) = 4     # gcc -O0
max2(4, 9) = 0     # gcc -O2
max2(4, 9) = 32525 # clang 18
```

HA3 · dönüş değeri kullanılmamış

Hatalı: sonuçlar kayboluyor

hata_avcisi/H08_HA3_donus_kullanilmiyor.c · satır 14–20

```
14      char harf = 'k';
15      toupper(harf);                  /* sonuç kayboldu */
16      printf("Büyük harf: %c\n", harf);
17
18      long long sonuc = 1;
19      us_al(2, 10);                   /* sonuç kayboldu */
20      printf("2^10 = %lld\n", sonuc);
```

Düzeltilmiş: dönüş değeri saklanıyor

hata_avcisi/H08_HA3_donus_kullanilmiyor_duzeltilmis.c · satır 13–18

```
13      char harf = 'k';
14      harf = toupper(harf);           /* dönüş değeri saklanıyor */
15      printf("Büyük harf: %c\n", harf);
16
17      long long sonuc = us_al(2, 10);
18      printf("2^10 = %lld\n", sonuc);
```

`toupper` ve `us_al` argümanı **değiştirmez**, yeni değer **döndürür**.

HATALI · GERÇEK ÇIKTI

```
$ ./ha3
Büyük harf: k
2^10 = 1
```

GCC 13.3 İLETİLERİ (KISALTILMIŞ)

```
toupper(harf);
warning: statement with no effect
us_al(2, 10);
(uyarı YOK)
C23 [[nodiscard]] + -std=c2x:
ignoring return value of ‘us_al’
```

DÜZELTİLMİŞ · GERÇEK ÇIKTI

```
$ ./ha3_duzeltilmis
Büyük harf: K
2^10 = 1024
```

Mühendis gözüyle · Merak kutusu

Bir fonksiyon, bir iş

Ne yaptığını “ve” demeden tek cümleyle söyleyin. Hesap yapan fonksiyon yazmasın, okumasın: **yan etkisiz** fonksiyon kolay test edilir.

İmza ve sözleşme belgedir

`int para_yatir(int bakiye)` girdiyi ve çıktıyı söyler; global değişken bunu saklar (H9). Her ön koşul sınırı bir **sınır testidir**.

Tanımsız davranışa sıfır tolerans

Taşabilecek ifadeyi yeniden yaz (`d <= n / d`); her yol `return`’e ulaşsın; uyarıları hata gibi oku (`-Werror`).

Yeniden düzenlemede davranış korunur

Örnek 13 ve Mini-OBS v0.8’de iki sürümü `diff` ile karşılaştırdık: endüstride **regresyon testleri**.

Merak: küçük fonksiyonun bedeli?

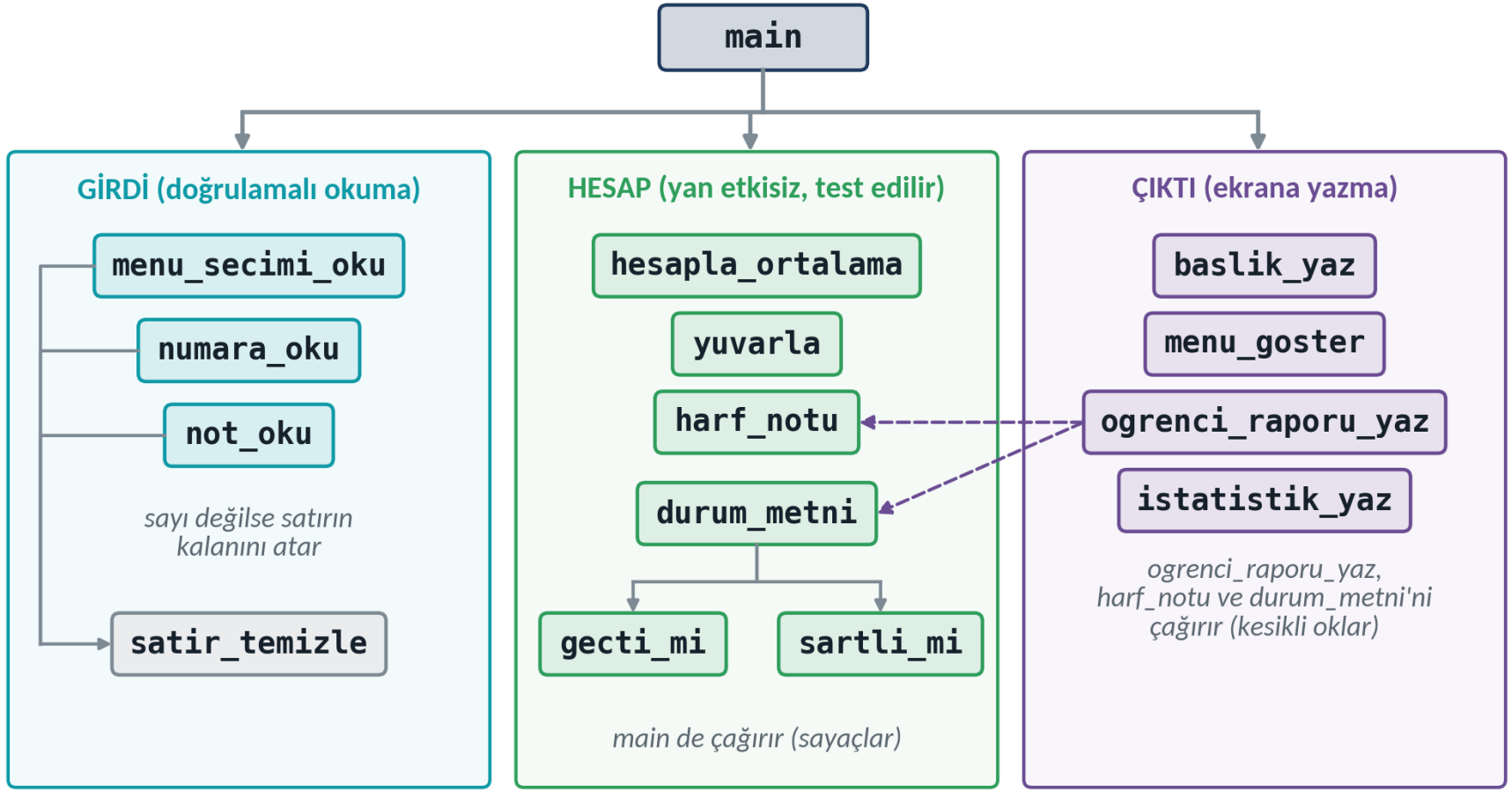
Her çağrı `call` + `ret` + çerçeve; ama `-O2` ile `topla` satır içine alındı, hiç çağrılmadı. Ölçmeden “yavaştır” demeyin.

Merak: argümanlar hangi sırayla?

`f(g(), h())`’de önce hangisinin çalışacağını C standardı **belirtmez**; iç içe çağrıda (`f(g(1))`) ise sıra kesindir.

Mini-OBS v0.8: fonksiyonlara ayrıştırma ve assert testleri

Mini-OBS v0.8 yapı şeması: main ve üç sorumluluk grubu (mini_obs_v0_8.c)



- `-DOBS_TEST`: **37 assert** geçti
- Bozuk kopya (BA `> 81`): satır 350'de yakalandı, çıkış **134**
- T1–T5: v0.7 ile `diff` → **yalnızca başlık**
- Hesap fonksiyonları `printf`/`scanf` çağırmaz
- `istatistik_yaz`: 8 parametre → H9'da `static`

AP-2 KÖPRÜSÜ

Her algoritma bir fonksiyondur

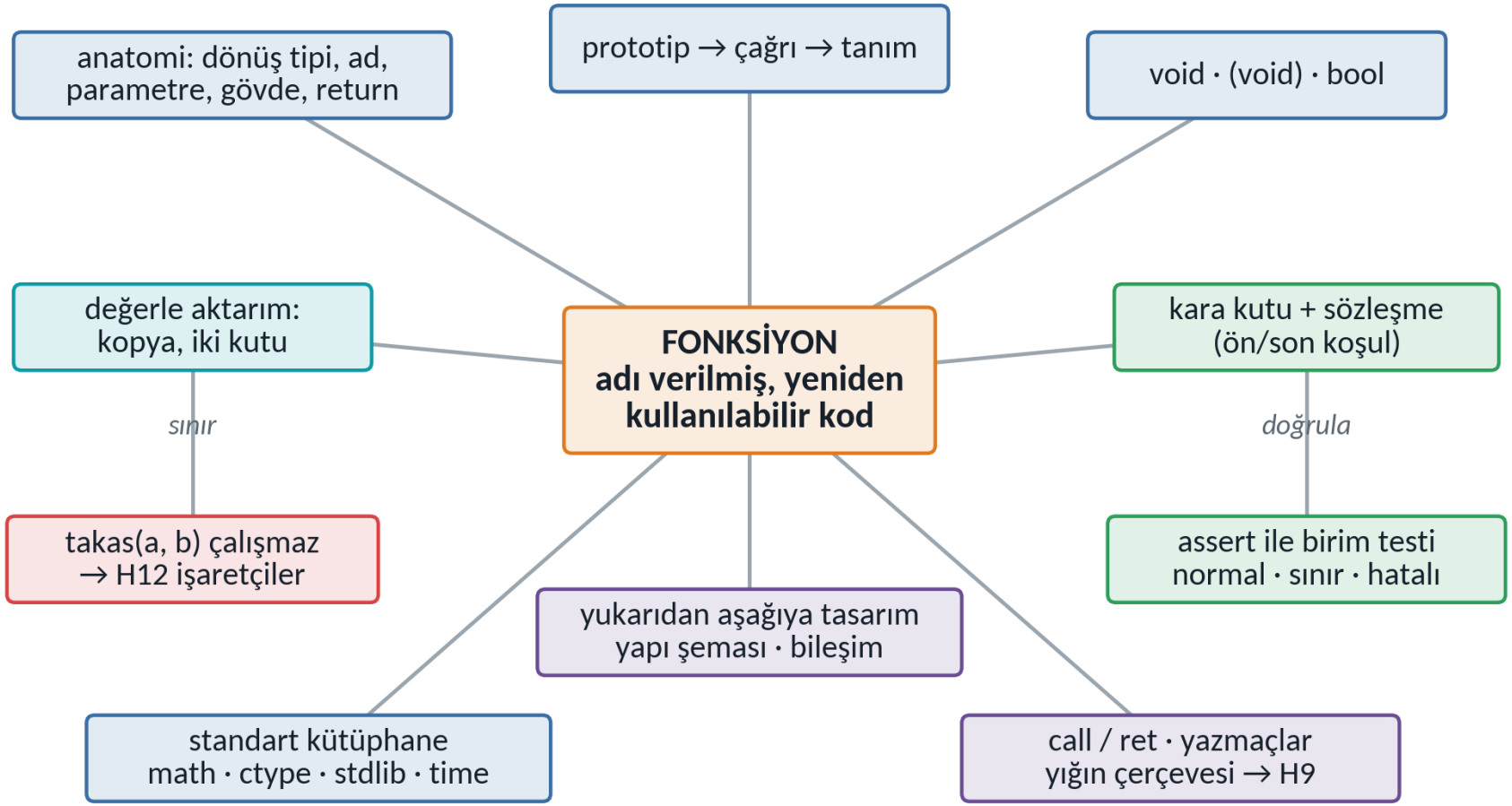
- AP-2'de arama, sıralama, EBOB, üs alma: her algoritma bağımsız, test edilebilir bir **fonksiyon**
- Sözleşme ve `assert` → bir algoritmanın **doğruluğunu** göstermenin ilk adımı
- Aynı sözleşmeyi karşılayan iki gerçekleştirim → **verimlilik** karşılaştırması
- Sıralamanın temel işlemi **takas**: Örnek 10'daki başarısız `takas(a, b)` → işaretçi (H12), dizi elemanları (H10)

K02: aynı sözleşme, iki gerçekleştirim (gerçek çıktı)

n	toplam_dongu	toplam_formul	döngü turu
0	0	0	0
10	55	55	10
100	5 050	5 050	100
100 000	5 000 050 000	5 000 050 000	100 000

Sonuçlar **aynı**, iş miktarı **farklı**: döngü n tur döner; formül (Gauss) sabit sayıda işlem yapar. Çağıran için ikisi aynıdır (kara kutu); hangisinin daha iyi olduğu AP-2'nin **verimlilik** sorusudur.

Özet: haftanın kavram haritası



- **Büyük Sorunun yanıtı:** yazılım, adı ve sözleşmesi tanımlı **parçalara** bölünür
- Parçayı kullanan yalnızca sözleşmeyi bilir (**soyutlama**)
- Her parça kendi **testleriyle** doğrulanır
- ATM: davranış aynı, `diff` ile **kanıtlandı**

ÇIKIŞ BİLETİ

Dersten çıkmadan önce üç soru

1 **Çıktı tahmini:** `void f(int x) { x = x + 10; }` ve `main`'de `int a = 5; f(a); printf("%d", a);` → ne yazılır? Neden?

2 **Kavram:** `int f();` ile `int f(void);` arasındaki fark nedir? Hangisini yazmalıyız?

3 **Hata bul:** `int max2(int a, int b) { if (a > b) { return a; } }` — derleyici ne der, `max2(4, 9)` neden güvenilmezdir?

Haftaya hazırlık ve kaynaklar

- **Oku:** ders notu Bölüm 3–5 (fonksiyon, sözleşme, kütüphane, assert, Hata Avcısı)
- **Web:** çağrı animasyonu (takas dahil), “Fonksiyonu tamamla” alıştırmaları, standart kütüphane başvuru kartı
- **Alıştırmalar:** 8.1–8.6 atölyede; 8.7–8.11 ödev (`kodlar/alistirmalar/`)
- **Mini-OBS:** v0.8’i `-DOBS_TEST` ile çalıştır; kendi `assert`’ini ekle
- **H9 önizleme:** kapsam ve ömür (`static`), çağrı yığını, özyineleme, Hanoi; hazırlık: Python Tutor’da `H08_K06_cagri_akisi.c`
- **Kaynaklar:**
  - Wirth, N. (1971). Program development by stepwise refinement. CACM, 14(4) · ISO/IEC 9899:2018
  - Bryant & O’Hallaron (2016). Computer Systems (3rd ed.) · Guo (2013, 2021). Python Tutor · Adler (1998); Martin (1994), ALSJ